

Role-oriented Network Embedding Based on Adversarial Learning between Higher-order and Local Features

Wang Zhang

Xuan Guo

wangzhang@tju.edu.cn
guoxuan@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Ting Pan

College of Intelligence and
Computing, Tianjin University
Tianjin, China
tingpan@tju.edu.cn

Chaochao Liu

Chinese Academy of Cyberspace
Studies
Beijing, China
chaochaoli@tju.edu.cn

Pengfei Jiao

Center of Biosafety Research and
Strategy, Law School, Tianjin
University
Tianjin, China
pjiao@tju.edu.cn

Lin Pan*

School of Marine Science and
Technology, Tianjin University
Tianjin, China
linpan@tju.edu.cn

Wenjun Wang

College of Intelligence and
Computing, Tianjin University
Tianjin, China
wjwang@tju.edu.cn

ABSTRACT

Roles of nodes are defined as classes of equivalent nodes. Nodes that have similar local connective patterns may share the same role. As a complementary concept of community, role can also help to recognize real-world entities. For example, it can denote identity or function in social networks. Role has been studied over the past decades, and learning role-based network representations is crucial to many downstream tasks. The important step for role-based network embedding method is extracting features to measure structural similarity instead of proximity. Although some methods have been developed to capture role features to learn structural similarities between nodes, they all design these features of fixed types, such as the global, local, and higher-order features. These features can only represent a certain type of structure, and it is very difficult to model the complex relationship between different scale features in the field of role-based network embedding. Therefore, we propose a novel role-oriented network embedding framework based on adversarial learning between higher-order and local features (ARHOL) to generate powerful role-based node representations. The higher-order features are discrete so we leverage the Auto-Encoder on them to obtain continuous representations. Then we apply the GIN on its outputs to aggregate local information. Finally, we consider the GIN as the generator and design an adversarial game between local features and GIN outputs to integrate these two aspects of features, which can enhance each other and improve the robustness. The extensive experiments on real-world networks

demonstrate the superiority and efficiency of our model, and prove the effectiveness of integrating higher-order and local features.

CCS CONCEPTS

• **Computing methodologies** → **Learning latent representations**; • **Information systems** → *Data mining*.

KEYWORDS

role-based network embedding, higher-order information, local features, adversarial learning

ACM Reference Format:

Wang Zhang, Xuan Guo, Ting Pan, Chaochao Liu, Pengfei Jiao, Lin Pan, and Wenjun Wang. 2021. Role-oriented Network Embedding Based on Adversarial Learning between Higher-order and Local Features. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM '21)*, November 1–5, 2021, Virtual Event, QLD, Australia. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3459637.3482051>

1 INTRODUCTION

Graphs or networks are irregular data structures, where nodes denote arbitrary entities and edges among them represent their relationships. However, these structures are usually discrete and cannot be used directly. The most efficient method for capturing information from networks is network embedding (also known as network representation learning), aiming to map the discrete representations of nodes into a continuous and low-dimensional vector space [6], which can be applied to many downstream tasks. Most of the existing network embedding methods are designed for community detection, such as DeepWalk [13] and node2vec [4]. They preserve the node proximity in the embedding space [15] and map nodes that are closely connected into similar vectors.

However, only a few studies concentrate on structural similarities. As a complementary concept of community [15], role of nodes is defined as class of equivalent nodes [11]. It describes the similarities of local connective patterns between nodes. Intuitively, the fundamental difference between role and community is that

*Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '21, November 1–5, 2021, Virtual Event, QLD, Australia

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8446-9/21/11...\$15.00

<https://doi.org/10.1145/3459637.3482051>

nodes in the same role may be not directly connected and can be distant from each other. To generate role-oriented node representations, the most important step is to extract structural features. Some methods have been developed to capture role features to learn structural similarities between nodes, which all design the features of fixed types, such as the global, higher-order and local features. Among these features, counts of motif patterns are useful to capture higher-order structure [1]. They list out all the patterns of arbitrary size of nodes, but when the size increases, counting these patterns becomes unrealistic. Graphlet Degree Vector (GDV) [9] can effectively represent higher-order information with small size of nodes. It enumerates 30 graphlets and 73 orbits of automorphism nodes. For each node, GDV generates a 73-dimensional vector, which can be regarded as its structural features. However, the size of 5 has limited power and loses global structural information.

There are also some methods capturing local[8] features, but they only capture one-sided features, which represent single type of roles. We believe that integrating these two aspects of features (higher-order and local features) can be helpful for role discovery. However, it is very difficult to model the complex relationship between different scale features in the role-based network embedding, and simply combining them is meaningless. Therefore, we propose a novel role-oriented network embedding framework based on adversarial learning between higher-order and local features (**ARHOL**) to integrate them. We leverage the Auto-Encoder (AE) to reconstruct the higher-order features, and map them into a continuous space. Meanwhile, these features lose global information, so we use the GIN [17] to aggregate neighbors. Finally, we design an adversarial game to integrate higher-order information and local features to generate more expressive representations.

2 METHOD

2.1 Notions

We will first introduce the notions. Given a network $G = \{V, E\}$ with adjacency matrix $A \in \mathbb{R}^{n \times n}$, where $V = \{v_1, v_2, \dots, v_n\}$ is the node set and $E \subseteq V \times V$ is the set of edges between nodes. The neighbors of node v_i is denoted as $N(i) = \{v_j | (v_i, v_j) \in E\}$, and $d(v_i)$ denotes the degree of v_i . We use $X \in \mathbb{R}^{n \times d_1}$ to denote the local features of nodes, while $M \in \mathbb{R}^{n \times d_2}$ denotes the higher-order features, where $n = |V|$ is the number of nodes and d_1, d_2 are the dimension of features respectively. The goal of our model is to design a function $f : f(X, M) \rightarrow \mathbb{R}^d$ to map the features into embedding space while preserving node structural similarity, where d is the dimension of node representation.

2.2 Model

As shown in Fig. 1, there are three parts in our model. We first extract higher-order features via GDV [9] and local features via ReFeX [8]. Then we use auto-encoder to reconstruct higher-order features to generate node representations. Finally, we leverage the GAN [3] structure and use GIN [17] as generator. We train it with adversary mechanism to integrate higher-order and local features.

2.2.1 Auto-encoder on higher-order features. As explained in Sec. 1, we use GDV [9] to capture higher-order features, because it counts

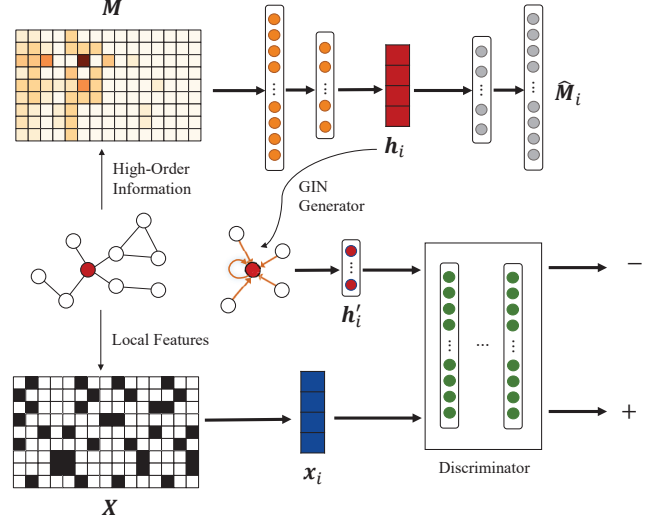


Figure 1: The illustration of our model ARHOL.

graphlets with the max size of 5 nodes and can be applied on large-scale networks. However, each orbit or dimension in these features is discrete and independent, which is not suitable to be directly used for downstream tasks. So we leverage the auto-encoder framework to reconstruct the features and generate node representations consisting of higher-order information.

We use multi-layer perceptrons (MLP) to construct the auto-encoder. Given the higher-order feature vector $m_i \in \mathbb{R}^{d_2}$ of node v_i , we have:

$$h_i^{(l)} = \text{ReLU}(\mathbf{W}^{(l)} h_i^{(l-1)} + \mathbf{b}^{(l)}), l = 1, 2, \dots, L, \quad (1)$$

where L is the number of layers. $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weight matrix and bias of the l -th layer respectively. $h_i^{(l)}$ is hidden representation of node v_i , and we have $h_i^{(0)} = m_i$. We use the output of the last encoder layer $h_i = h_i^{(L)}$ as the final node representation.

For the part of decoder, we use h_i to reconstruct its higher-order features:

$$\hat{h}_i^{(l-1)} = \text{ReLU}(\hat{\mathbf{W}}^{(l)} \hat{h}_i^{(l)} + \hat{\mathbf{b}}^{(l)}), l = L, L-1, \dots, 1, \quad (2)$$

where $\hat{h}_i^{(l)} = h_i$, while $\hat{\mathbf{W}}^{(l)}$ and $\hat{\mathbf{b}}^{(l)}$ are the weight matrix and bias of the l -th decoder. $\hat{m}_i = \hat{h}_i^{(0)}$ is the output of decoder. We define the loss of AE as:

$$\mathcal{L}_{AE} = \|M - \hat{M}\|_2^2 = \sum_{v_i \in V} \|m_i - \hat{m}_i\|_2^2. \quad (3)$$

2.2.2 Adversarial learning. Although GDV can capture higher-order information, it cannot obtain accurate results because of the limited size of 5 nodes. And the higher-order features lose global information of the whole network. So we leverage the ReFeX features to enhance the above auto-encoder. We design a generator[17] and a discriminator with MLP, as GIN has been proved for its great power of capturing structural similarity. The GIN generator is defined as

follows:

$$\mathbf{h}_i^{(k)} = \text{MLP}^{(k)}((1 + \epsilon^{(k)})\mathbf{h}_i^{(k-1)} + \sum_{v_j \in N(i)} \mathbf{h}_j^{(k-1)}), \quad (4)$$

where $\mathbf{h}_i^{(k)}$ is hidden representation of the k -th GIN layer. $\epsilon^{(k)}$ is a trainable parameter to distinguish the center node and neighbors. Each layer of GIN consists of independent multi-layer perceptrons, and with the number of layers increases, it can capture global information.

We also leverage MLP to design the discriminator, the goal of which is to output a single value distinguishing positive samples (from ReFeX features) and negative samples (from GIN generator). We define it as follows:

$$D(\mathbf{x}) = \text{softmax}(\text{MLP}(\mathbf{x})), \quad (5)$$

where $D(\cdot)$ denotes the discriminator. The softmax activation controls the output between 0 and 1, indicating the probability of positive sample.

We use θ_D to denote the parameters of discriminator, and the whole generator is denoted as $\Phi(\mathbf{x})$. For a given node v_i , the generator attempts to capture global information based on the output of auto-encoder, while the discriminator will try to detect whether the representations are true local features or not. The loss function of the adversarial structure is defined as follows:

$$\mathcal{L}_{\Phi, D} = \min_{\theta_{\Phi}} \max_{\theta_D} \sum_{i=1}^{|V|} [\log(D(\mathbf{x}_i)) + \log(1 - D(\Phi(\mathbf{h}_i)))]. \quad (6)$$

2.2.3 Optimization. We separately optimize the generator and discriminator. The goal of discriminator is to maximize the distance between positive samples and negative samples. We use θ_D to denote the parameters of discriminator, and the whole generator is denoted as $\Phi(\mathbf{x})$. We have:

$$\theta_D = \arg \max_{\theta_D} \sum_{i=1}^{|V|} [\log(D(\mathbf{x}_i)) + \log(1 - D(\Phi(\mathbf{h}_i)))]. \quad (7)$$

By contrast, the generator is designed to fool the discriminator. For each iteration, it is trained by the feedback signal from the discriminator.

$$\theta_{\Phi} = \arg \min_{\theta_{\Phi}} \sum_{i=1}^{|V|} \log(1 - D(\Phi(\mathbf{h}_i))) \quad (8)$$

Finally, to integrate the higher-order information and local features, we jointly train the auto-encoder and the GIN generator as:

$$\mathcal{L} = \arg \min_{\theta_{AE}, \theta_{\Phi}} \sum_{i=1}^{|V|} \|\mathbf{m}_i - \hat{\mathbf{m}}_i\|_2^2 + \alpha \log(1 - D(\Phi(\mathbf{h}_i))), \quad (9)$$

where α is the hyper-parameter to control the weight of local features.

3 EXPERIMENTS

3.1 Experiment settings

We conduct extensive experiments on several real-world networks. The details are shown in Table 1.

Table 1: The statistical information of the six networks.

Dataset	# Nodes	# Edges	# Classes	Density (%)
Brazil	131	1,003	4	11.7792
Europe	399	5,995	4	7.5478
USA	1,190	13,599	4	1.9222
Actor	7,779	26,752	4	0.0886
Aminer-data	98,460	303,029	3	0.0063
Aminer-system	146,092	400,863	3	0.0038

Air-traffic networks [14]: There are three air-traffic networks Brazilian, European, and American networks (shortly denoted as Brazil, Europe, and USA). Nodes represent airports and edges represent flights between them. The labels are between 0 and 3 reflecting the levels of the airport activities.

Actor co-occurrence network (Actor) [12]: This is an actor only subgraph of a film-director-actor-writer network extracted from IMDb. The nodes denote actors and edges denote co-occurrence on the same Wikipedia page. Nodes are sorted based on the number of words on their pages and split into four groups.

Aminer cooperation networks¹: This is a large network consisting of more than 1 million nodes and 4 million edges. Nodes represent papers and edges represent their cooperation relationships. We extract two subgraphs based on authors' keyterms (shortly denoted as Aminer-data and Aminer-system), where the words "data" and "system" appear in their keyterms respectively. Nodes are labeled based on their h-indices.

We compare our model with the following role-based embedding methods: node2vec [4], struc2vec [14], RoLX [7], DRNE [16], GraphWave [2], RESD [18], and GAS[5]. The hyper-parameters are tuned to the best according to their papers, and the dimensions are set to 128. While for GraphWave, the dimension is set to 100 based on its paper.

In the following experiments, we set the number of iterations to 3 and bins to 4 for ReFeX features. While for GDV, we use the size of 5 nodes, and we obtain 73-dimensional features. For our proposed model ARHOL, we set the number of layers to 3 for each component, and in the GIN generator, we use ReLU as the activation function. In the training process, we apply Adam SGD optimizer [10] to update parameters with learning rate of 0.001. We set the weight of L_2 regularizer to 0.01 to avoid over-fitting. The max number of training epochs is set to 100, and the effect of hyper-parameter α is analyzed in Sec. 3.5.

3.2 Role classification

We conduct the task of role classification on six real-world networks. The nodes are labeled based on their identities or functions. We first generate node representations and then choose the linear logistic regression classifier to classify nodes. We randomly choose 10% to 90% samples to train the classifier and use the remain to test. We repeat the above process for 20 times and report their F1 scores. As shown in Table 2, for the small networks (Brazil, Europe, USA, and Actor), we find RESD gets the best results on Brazil, because

¹Available at <https://www.aminer.cn/panther>

Table 2: Accuracy of classification. ① to ⑥ denote Brazil, Europe, USA, Actor, Aminer-data, and Aminer-system respectively. OOM denotes the out of memory error.

Method	①	②	③	④	⑤	⑥
node2vec	42.92	39.79	49.64	35.17	42.44	43.23
struc2vec	77.47	57.04	59.66	46.49	49.44	50.92
RoIX	72.77	54.46	62.92	46.81	52.22	52.45
DRNE	71.03	51.58	55.24	43.17	45.84	47.77
GraphWave	76.97	52.62	52.17	47.91	OOM	OOM
RESL	78.92	57.47	64.80	47.87	54.48	54.55
GAS	73.65	54.00	65.08	48.27	OOM	OOM
ARHOL	77.88	60.53	66.03	48.93	57.10	57.07

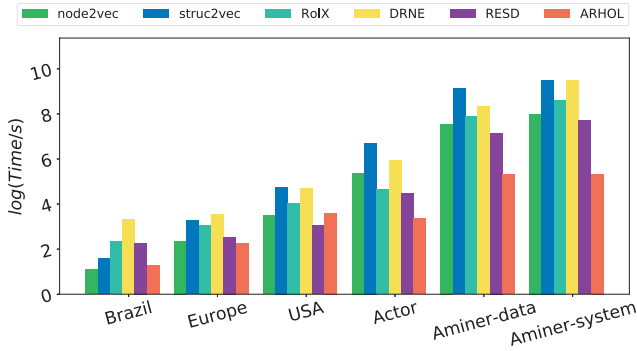


Figure 2: Time of generating node embeddings on different networks. The horizontal axis denote different datasets, and the vertical axis is the logarithmic time.

the size of this network is too small and local features are powerful enough to represent roles. But we also observe that when we use 90% nodes to train the classifier, our model ARHOL performs best. While on the other three networks, ARHOL is still the best model. As the large scale networks (two Aminer networks), our model gets significant improvement.

3.3 Efficiency analysis

We compare the time for generating node embeddings to analyze the efficiency of these methods. As shown in Fig. 2, the network size increases on the horizontal axis. Note the results of GraphWave and GAS are absent, because they cannot generate embeddings on the two Aminer networks because of the out of memory error. We observe that when the size of network increases, the deep-learning models show their advantages. Our model is the fastest among all methods, which highlights the efficiency of our model, and demonstrates that ARHOL can be applied on large-scale networks.

3.4 Ablation Study

We randomly choose 70 percent of samples to train a linear classifier and use the remain to test. We repeat this step 20 times and report the mean accuracy and standard deviation in Table 3. The results demonstrate that simply combining two aspects of features

Table 3: Ablation study of ARHOL on three air-traffic networks. GDV and ReFeX denotes two kinds of features, while AE denotes that we only use the auto-encoder to reconstruct higher-order features without adversarial learning. The operator + denotes the combination of dimension.

Model	Brazil	Europe	USA
GDV	70.30 $\pm$ 6.13	60.01 $\pm$ 3.83	65.43 $\pm$ 2.22
GDV+ReFeX	75.63 $\pm$ 6.04	56.21 $\pm$ 3.37	64.77 $\pm$ 2.11
AE	73.58 $\pm$ 5.18	58.72 $\pm$ 3.57	65.45 $\pm$ 2.23
AE+ReFeX	75.18 $\pm$ 5.55	57.60 $\pm$ 3.70	64.45 $\pm$ 2.01
ARHOL	77.88 $\pm$ 5.61	60.53 $\pm$ 3.87	66.03 $\pm$ 2.11

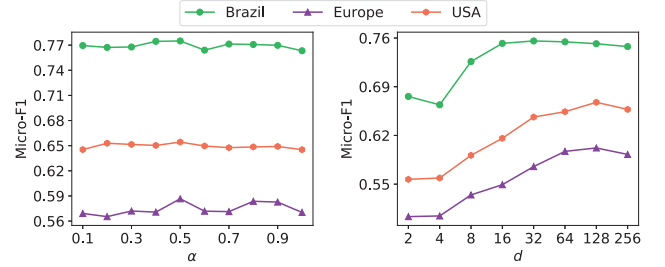


Figure 3: The analysis of the weight of local features α and embedding dimension d

obtains poor performance, and our adversarial game can effectively integrate higher-order and local features.

3.5 Parameter analysis

In this subsection, we report the analysis of two hyper-parameters: the weight of local features in the adversarial game and the embedding dimension d . In our experiments, we fix one parameter and adjust the other one. We choose three small networks to show the results, and other datasets show a similar tendency. As shown in the left figure in Fig. 3, when $\alpha < 0.5$, the classification results increase with α increasing. We obtain the best results with $\alpha = 0.5$. As for the embedding dimension d , when $d \geq 64$, ARHOL obtains good enough results. This experiments indicates that the high-order information alone cannot accurately capture structural similarities, and local features can indeed assist it.

4 CONCLUSION

We propose a novel adversarial framework called ARHOL, which is designed for role-based network embedding. It leverages the auto-encoder on higher-order features and the GIN on its outputs to aggregate neighbors. Then we design an adversarial game with local features to improve robustness. Extensive experiments demonstrate the superiority of the model.

ACKNOWLEDGMENTS

This work is supported by the National Natural Science Foundation of China (61902278).

REFERENCES

- [1] Austin R Benson, David F Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.
- [2] Claire Donnat, Marinka Zitnik, David Hallac, and Jure Leskovec. 2018. Learning structural node embeddings via diffusion wavelets. In *SIGKDD*. 1320–1329.
- [3] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial networks. In *NIPS*.
- [4] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *SIGKDD*. 855–864.
- [5] Xuan Guo, Wang Zhang, Wenjun Wang, Yang Yu, Yinghui Wang, and Pengfei Jiao. 2020. Role-Oriented Graph Auto-encoder Guided by Structural Information. In *DASFAA*. 466–481.
- [6] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation Learning on Graphs: Methods and Applications. *IEEE Data Engineering Bulletin* 40, 3 (2017), 52–74.
- [7] Keith Henderson, Brian Gallagher, Tina Eliassi-Rad, Hanghang Tong, Sugato Basu, Leman Akoglu, Danai Koutra, Christos Faloutsos, and Lei Li. 2012. Rolx: structural role extraction & mining in large graphs. In *SIGKDD*. 1231–1239.
- [8] Keith Henderson, Brian Gallagher, Lei Li, Leman Akoglu, Tina Eliassi-Rad, Hanghang Tong, and Christos Faloutsos. 2011. It's who you know: graph mining using recursive structural features. In *SIGKDD*. 663–671.
- [9] Tomaž Hočevar and Janez Demšar. 2014. A combinatorial approach to graphlet counting. *Bioinformatics* 30, 4 (2014), 559–565.
- [10] Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *ICLR*.
- [11] Francois Lorrain and Harrison C White. 1971. Structural equivalence of individuals in social networks. *The Journal of mathematical sociology* 1, 1 (1971), 49–80.
- [12] Xuewei Ma, Geng Qin, Zhiyang Qiu, Mingxin Zheng, and Zhe Wang. 2019. RiWalk: fast structural node embedding via role identification. In *ICDM*. 478–487.
- [13] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *SIGKDD*. 701–710.
- [14] Leonardo FR Ribeiro, Pedro HP Saverese, and Daniel R Figueiredo. 2017. struc2vec: Learning node representations from structural identity. In *SIGKDD*. 385–394.
- [15] Ryan A Rossi, Di Jin, Sungchul Kim, Nesreen K Ahmed, Danai Koutra, and John Boaz Lee. 2020. On proximity and structural role-based embeddings in networks: Misconceptions, techniques, and applications. *TKDD* 14, 5 (2020), 1–37.
- [16] Ke Tu, Peng Cui, Xiao Wang, Philip S Yu, and Wenwu Zhu. 2018. Deep recursive network embedding with regular equivalence. In *SIGKDD*. 2357–2366.
- [17] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *7th International Conference on Learning Representations*.
- [18] Wang Zhang, Xuan Guo, Wenjun Wang, Qiang Tian, Lin Pan, and Pengfei Jiao. 2021. Role-based network embedding via structural features reconstruction with degree-regularized constraint. *KBS* (2021), 106872.